

AI Agent for Dimension Reduction and Exploratory Data Analysis

Cassi Chen

1 Introduction and Overview of System Architecture

Dimension-reduction (DR) analyses and exploratory data analysis (EDA) require decisions for preprocessing, method selection, hyperparameters, evaluation, and interpretation. Unlike a traditional benchmarking study, this project develops an AI agent that automatically performs DR and EDA with minimal human intervention. Given a dataset, the agent not only inspects the data, preprocesses the data, chooses DR methods and its hyperparameters, evaluates and interprets the results, but also performs targeted follow-up and generates a final analysis report, all autonomously.

The overall agent design in this project was inspired by the UNC BIOS 774 course lecture 9 slides [1]. Specifically, the researcher sets objectives, the local code performs deterministic computations and saves metrics and results, and the LLM recovers context and invokes the code and explains the result against the task.

Figure 1, shows the model architecture. Codex (GPT-6 Astra Light) acts as the LLM reasoning agent that invokes code and interprets results. Codex uses `AGENTS.md` and `SKILL.md` as its main set of instructions, where `AGENTS.md` defines the overall workflow and `SKILL.md` files provide details for data inspection, preprocessing, DR methods, and embedding evaluation. Codex uses these skills to determine what methods are appropriate and how to interpret their outputs. Local Python programs provide tools for the data inspection, preprocessing, DR, and embedding evaluation. These are all deterministic computations and save metrics and results. The Python tools support three preprocessing choices: None, feature-wise standardization, and library-size normalization followed by $\log_{1p}$ transformation. The available DR tools the agent can use are Isomap, Kernel PCA, Laplacian Eigenmaps, LLE, MDS, PCA, t-SNE, and UMAP. The agent considers all DR methods but only selects a subset for the observed data.

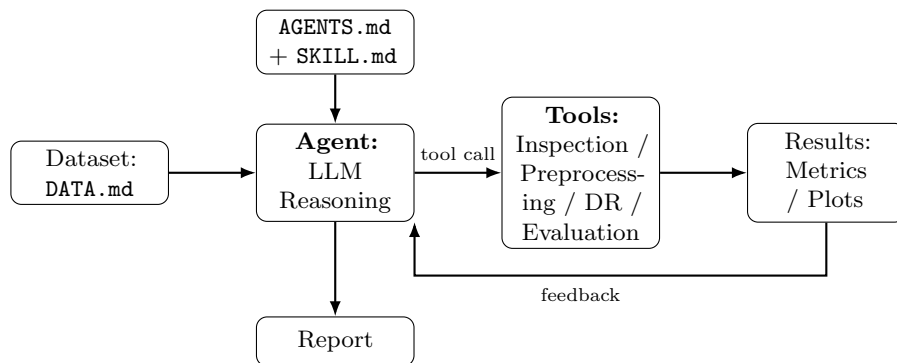


Figure 1: Agent architecture.

2 Agent Decision-Making and Implementation

The agent begins by invoking the data-inspection tool, which describes the number of features, missing values, sparsity, feature types, and other relevant details. Based on these details, the agent will choose a preprocessing method.

After preprocessing, the agent considers all the available DR methods. The agent selects DR methods based on the results from the data inspection skill, original data structure, method assumptions, and computational requirements. Hyperparameters are chosen by the agent, where reasonable default values are provided. The agent may perform a small targeted sensitivity analysis if necessary.

If the DR method supports out-of-sample transformation, the models are first fitted on the training data and then used to transform evaluation data. Note that Kernel PCA, LLE, PCA, Isomap, and UMAP support training and evaluation data. MDS, Laplacian Eigenmaps, and t-SNE support training data only. Labels are used afterwards for visualization and interpretation.

Embedding evaluations are based on DR method-specific and shared diagnostics. For example, PCA reports explained and cumulative variance, while trustworthiness provides describes local-neighborhood preservation across all DR

methods.

The main tools and AI models used in this project is provided in Table 1.

Component	Type	Role
Codex (GPT-6 Astra Light)	AI Model	Tool selection, interpretation, and reporting
<code>inspect_data.py</code>	Tool	Dataset inspection
<code>preprocessing.py</code>	Tool	Preprocessing transformations
<code>run_pca.py</code>	Tool	PCA fitting and diagnostics
<code>run_kernelpca.py</code>	Tool	Kernel PCA fitting and embeddings
<code>run_mds.py</code>	Tool	MDS fitting and diagnostics
<code>run_isomap.py</code>	Tool	Isomap fitting and diagnostics
<code>run_lle.py</code>	Tool	LLE fitting and diagnostics
<code>run_lapleigen.py</code>	Tool	Laplacian Eigenmaps embeddings
<code>run_tsne.py</code>	Tool	t-SNE fitting and diagnostics
<code>run_umap.py</code>	Tool	UMAP fitting and embeddings
<code>evaluate_embeddings.py</code>	Tool	Trustworthiness embedding evaluation

Table 1: AI model and deterministic Python tools available to the agent.

3 Experimental Results

The system was evaluated on two different datasets: PathMNIST [2] and PBMC3K [3]. Analysis on PathMNIST was conducted using a class-balanced subset of the original dataset, resulting in 4,500 training images consisting of 500 per class and 900 evaluation images consisting of 100 per class for the 9 classes. For PBMC3K, 2,638 of the original 2,700 cells had corresponding Seurat cell-type annotations and were retained. 2,110 cells were used for training and 528 were used evaluation. Gene filtering reduced the dataset to 2,000 genes. Summary of experimental results can be found in Table 2.

	PathMNIST	PBMC3K
Preprocessing	None	Log normalize
Selected methods	PCA, UMAP	PCA, UMAP
PCA 2D variance	57.01%	10.12%
PCA train trustworthiness	0.8059	0.7891
PCA eval trustworthiness	0.8475	0.7892
UMAP train trustworthiness	0.8141	0.7813
UMAP eval trustworthiness	0.8468	0.7792
UMAP Follow-up	$k = 30$	$k = 30$

Table 2: Summary of results.

In the following subsections, results focus on training data. Results and figures for evaluation data are omitted for brevity but are available in the generated reports and output directories. Evaluation embeddings were examined to assess whether broad structural patterns observed in training persisted in held-out data.

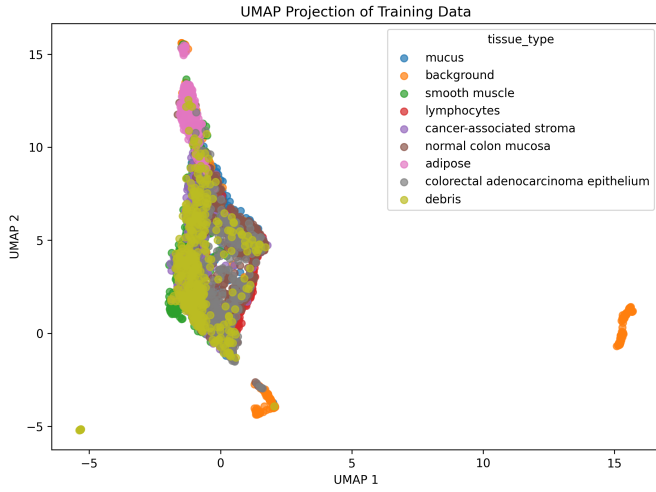
PathMNIST.

Each 28×28 image in PathMNIST was represented by 2,352 pixel features. Inspection showed that the pixels shared the same intensity scale and had comparable variability. Thus, the agent selected no additional preprocessing.

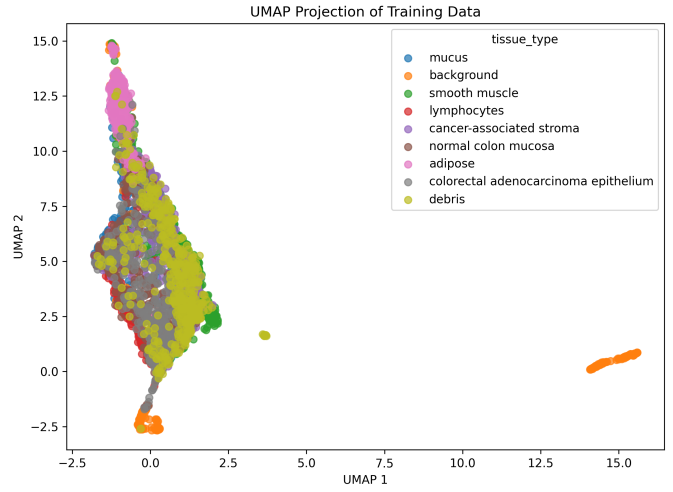
The first two PCs explained 57.01% of training variance 3(a), and PCA and UMAP had similar evaluation trustworthiness (0.8475 and 0.8468, respectively). Due to detached regions in the UMAP, the agent increasing `n_neighbors` from 15 to 30 while holding other hyperparameters constant to perform a targeted follow-up. Results show that the general broad structure remained, as seen in Figure 2(a-b). Overall, PCA and UMAP show overlap among tissue types, implying that there is not a clear tissue-class separation in two dimensions.

PBMC3K.

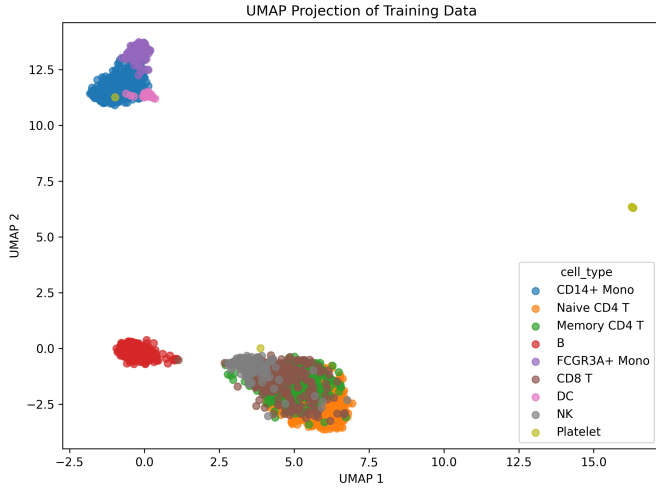
In contrast to PathMNIST, PBMC3K inspection showed sparse nonnegative counts and substantial variation in per-cell totals. Thus, the agent selected library-size normalization to 10,000 followed by `log1p`.



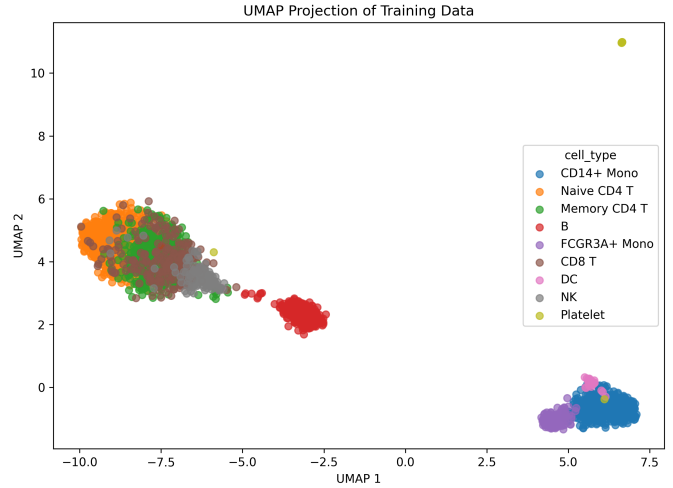
(a) PathMNIST, $k = 15$



(b) PathMNIST, $k = 30$



(c) PBMC3K, $k = 15$



(d) PBMC3K, $k = 30$

Figure 2: UMAP training embeddings before and after the agent’s targeted sensitivity analysis.

The first two PCs explained only 10.12% of training variance; thus, there is substantial information beyond two dimensions 3(b). Similarly to PathMNIST, PCA and UMAP for PBMC3K had similar evaluation trustworthiness (0.7892 and 0.7792, respectively). Also similarly to PathMNIST, due to the separated UMAP regions, the agent increased `n_neighbors` from 15 to 30, while holding other hyperparameters constant. Results show that the broad structure remained with only a major change in relative placement, as shown in Figure 2(c-d). In Both PCA and UMAP, some cells like the B-cell populations have a clear broad cell-type-associated structure. However among other cell types, there is still overlap.

4 Strengths, Limitations, and Conclusion

The main strength of the system is that entire process is autonomously done by an agent. The two tests conducted on PathMNIST and PBMC3K also show that the agent is able to adapt across different data modalities.

The system also has many limitations. First, the agent can only choose among the supplied preprocessing and DR methods. Second, preprocessing and DR methods and hyperparameter selection rely solely on LLM (Codex) reasoning instead of a specified criteria. Finally, trustworthiness measures only local-neighborhood preservation, while method-specific diagnostics assess distinct properties of the embeddings, making direct comparisons across metrics difficult.



Figure 3: PCA training embeddings.

Overall, this project showcases an AI agent that automatically performs EDA through DR, while maintaining reproducibility.

References

- [1] Longfei Zhang. BIOS 774 Course Slides. *An Intro to Building Agents in 2026*, 2026.
- [2] Jiancheng Yang, Rui Shi, and Bingbing Ni. MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10:41, 2023.
- [3] 10x Genomics. *PBMC 3K Dataset*. 10x Genomics.